

Table of Contents

1. [Dataset Preview • Overview • Palantir](#)
2. [Dataset Preview • Overview • Palantir](#)
3. [Dataset Preview • CSV parsing • Palantir](#)
4. [Dataset Preview • Time Travel • Palantir](#)
5. [Dataset Preview • FAQ • Palantir](#)

Dataset Preview

The Dataset Preview application provides you with a variety of details of a given dataset, including metadata, build history, health, and more. Additional features are available for streaming datasets, including the ability to view information on streaming jobs and metrics to troubleshoot and debug stream performance.

The screenshot below displays the interface of the Dataset Preview application. The numbered sections are explained in more details in the following sections.



1. Dataset header

The header of the page identifies the selected dataset and provides basic information such as its name, display name (if existing), location, and the selected branch. The header also allows some file-related operations such as sharing, moving, renaming, and more.

[API Reference ↗](#)

2. Information panel

[Send feedback](#)

The information panel provides metadata about the dataset and some basic administrative operations. The panel is divided into three sections:

- **About:** Information including the time the dataset was created and updated, the users who created and last updated the dataset, the size of the table, any tools and input datasets used to create the data, tags, and more. An **Edit schema** view is also available under the **Updated by** section that will infer a schema for CSV and JSON files. Here, users can also apply additional parsing options to drop jagged rows, change encoding, or add additional columns like file path, byte offset for row, import timestamp, or row number. For other file types, schema edits can be made in the **Schema** section under the **Details** tab.
- **Columns:** Information on the different columns in the dataset, including the type of data, description, and data stats (percentage of null values, distributions and samples).
- **Schedules:** Information about any configured build [schedules](#) that will run to update the dataset.

3. Tab views

Preview

The **Preview** tab provides a sample of the data in a table and allows light interaction with the full dataset. Learn more about the preview table in [4. Preview table](#).

History

The **History** tab view provides historical job (build) information. A **Summary** view on the right side of the page shows aggregated information on job statuses over time.

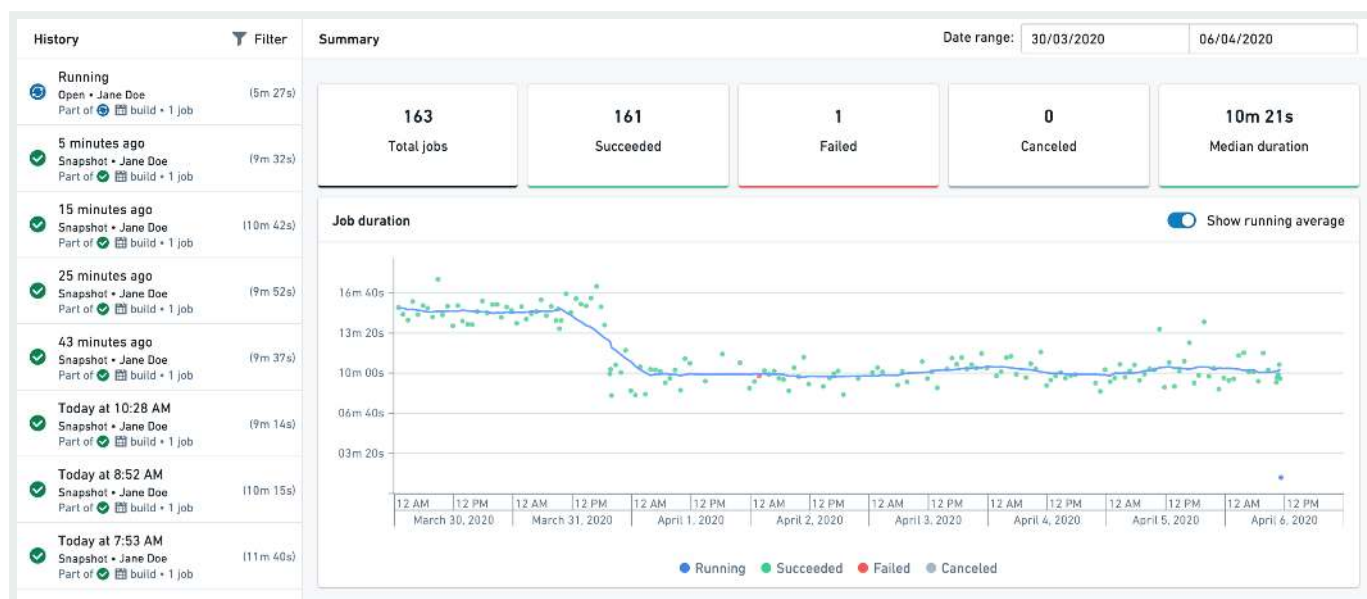
On the left panel, a list of jobs appears with their statuses and durations. Upon selection, a detailed **Job** view appears on the right showing detailed job information, including progress, specification, build logs, files and the resulting schema.

API Reference ↗

i Streaming datasets

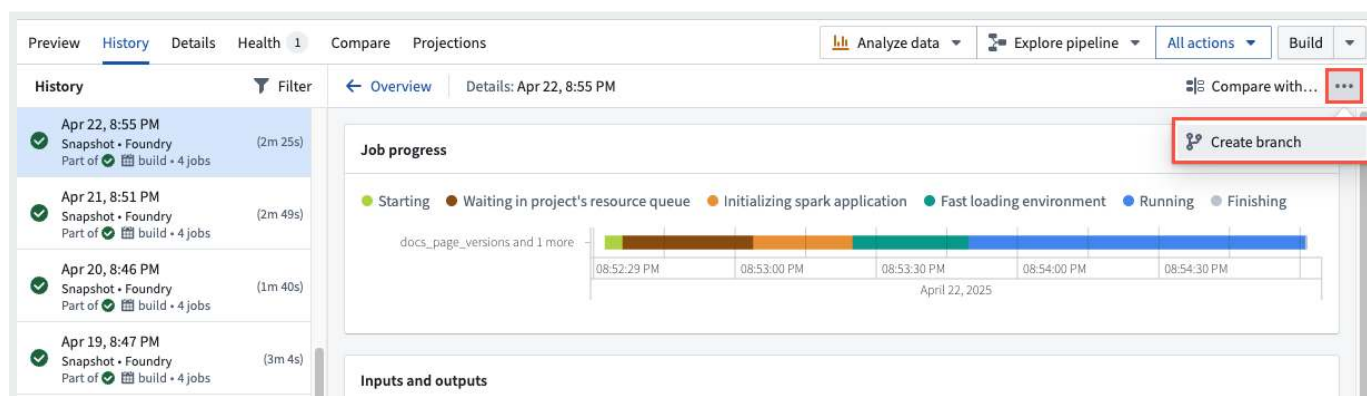
Send feedback

In streaming datasets, the **History** tab will only appear when the view is set to **Archive**. The **History** tab will show the archive transactions alongside the streaming jobs.



Create branches from historical transactions

You can use the **History** tab to create branches on historical transactions of your data that have not been deleted by a [retention policy](#). Choose a previous transaction from the left panel and select the ellipsis (...) icon to **Create branch**.



Details

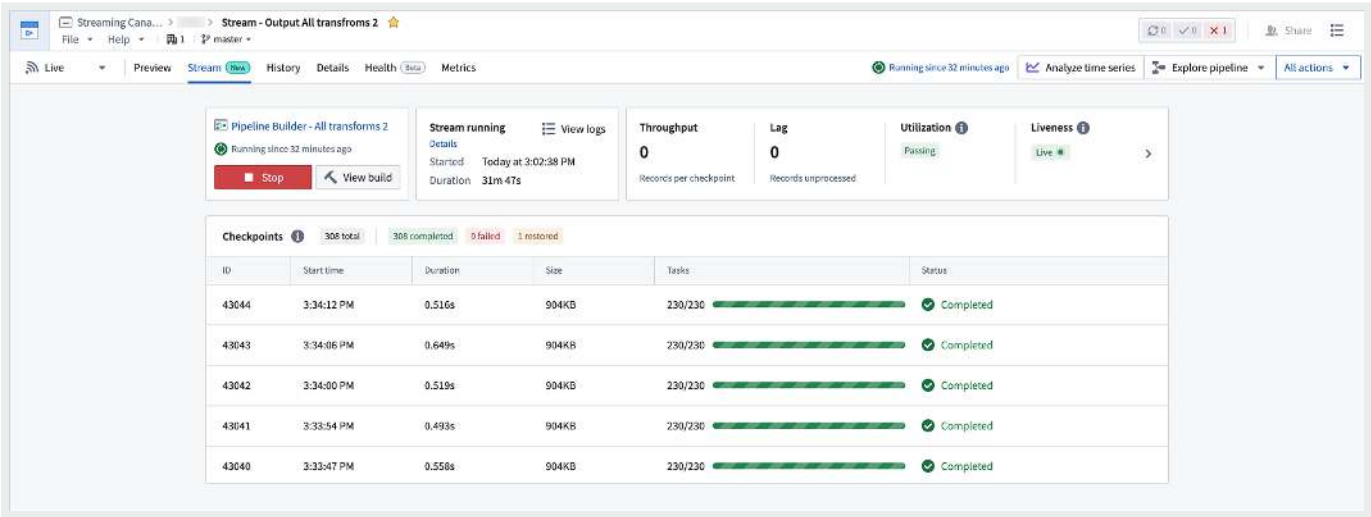
The **API Reference** provides additional technical information about the dataset, as well as some administrative operations:

[Send feedback](#)

- **Schema:** Provides full information on the table schema (column specifications) and allows you to edit the schema (if applicable).
- **Files:** Displays the list of files that make up the dataset and allows you to download them.
- **Job spec:** Displays the job specification containing essential information for the dataset to build.
- **Syncs:** Surfaces the status and details of data syncs to different databases. For some sync types, additional settings can be applied.
- **Custom metadata:** Allows you to add custom fields of information to the dataset. The fields added in this section are displayed in the information panel of the overview page.
- **Resource usage metrics:** Provides graphs and information on disk and Spark usage of the dataset over time.
- **Last run details (only for streams):** Shows detailed information about the latest stream run.

Stream (only for streaming datasets)

When the dataset is a streaming dataset, the **Stream** tab will show current and historical information on the streaming jobs. By changing the time period, you can explore the logs and details of jobs that streamed the dataset during that time.



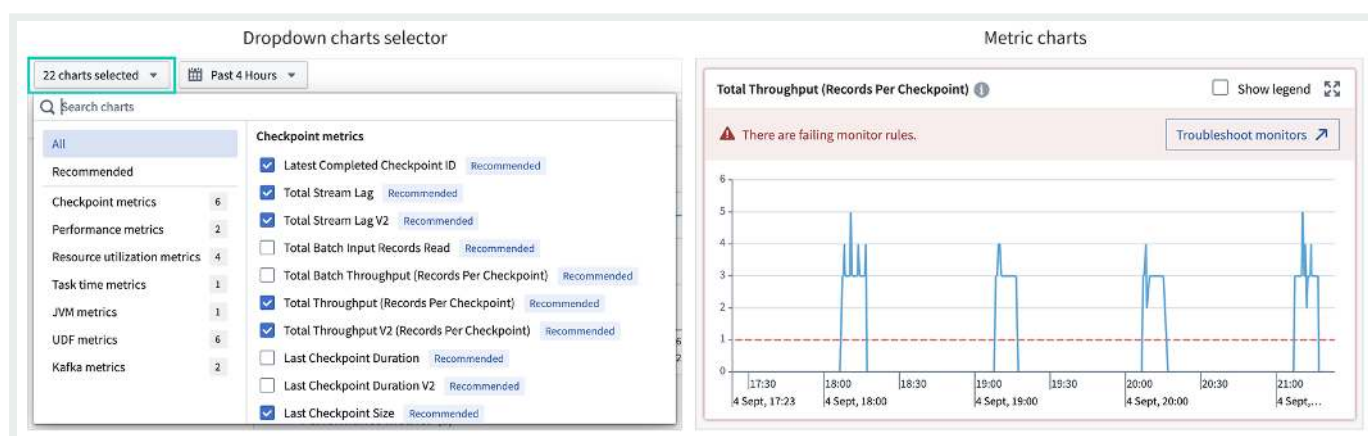
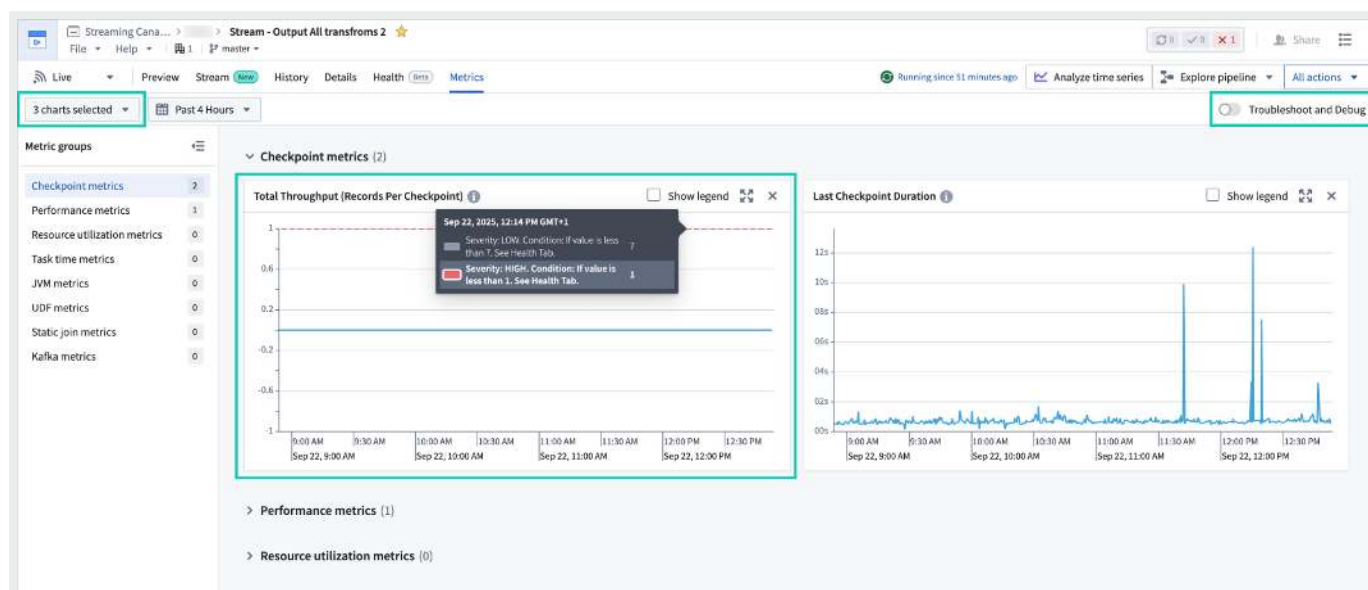
Metrics (only for streaming datasets)

API Reference ↗

When working with a streaming dataset, the Metrics tab shows charts and related interactions for analyzing and troubleshooting streaming job performance. A dropdown menu for selecting metrics to visualize trends, with recommendations

Send feedback

metrics highlighted within it. The metric charts can be expanded for detailed viewing and display thresholds and warning indicators with debug links.

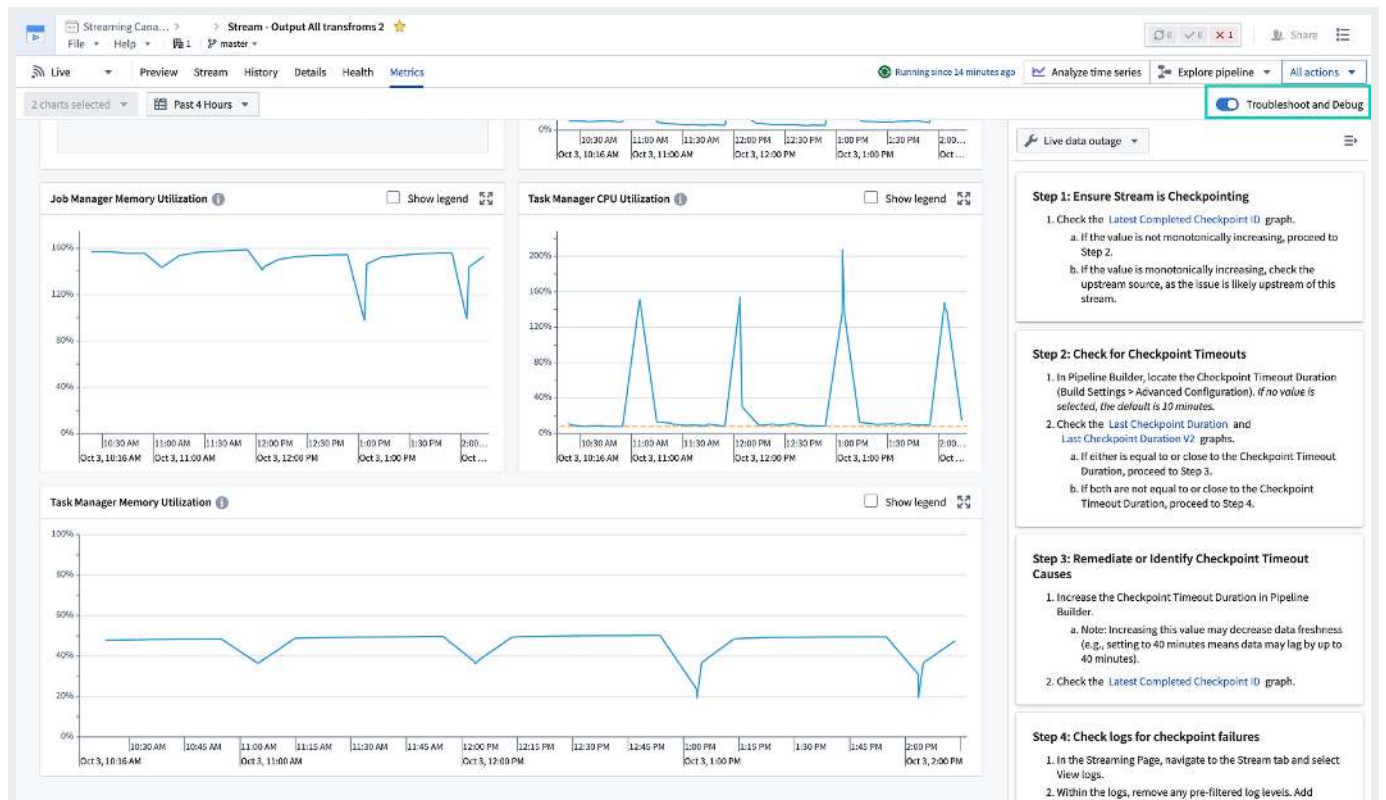


Troubleshoot and Debug mode

You can enable a dedicated troubleshooting mode using the toggle at the top right of the page. This mode provides a step-by-step walkthrough to debug stream outages. Select the in-line metrics tags from the right side panel to highlight the corresponding chart and easily locate the source of the issue.

API Reference ↗

Send feedback



Health

The **Health** tab provides tools to monitor [data health](#). The page displays health checks on the specific resource, monitoring rules on the resource grouped by specific monitoring view, and related schedule builds that affect the resource. Selecting any row reveals historical reports for the health checks and monitoring rules.

i Streaming datasets

In streaming datasets, the **Health** tab will only appear when the view is set to **Archive**. The checks will then refer to the archive dataset rather than the stream.

API Reference ↗

Send feedback

Pipelines > data > ... > nyc_yellow_taxi_trips_manhattan

File > Help > 1 master

Share

PreviewHistoryDetailsHealthCompare

Analyze dataExplore pipelineAll actionsBuild

☐ Show failures only

Search monitors...Add checks

Checks

Checks 321

NAME	STATUS	REPORTED AT	MONITORING VIEW	HISTORY REPORTS
Allowed column values: dropoff	Failed	Aug 14, 2025, 13:10	No monitoring view	●●●●●●●●●●
Data Expectations Monitor	Passed	Aug 14, 2025, 13:09	No monitoring view	●●●●●●●●●●
Dataset file count	1	Aug 14, 2025, 13:09	No monitoring view	●●●●●●●●●●

Monitoring

Foundry Test Passing (28) Failing (2)

Rules 9000

NAME	STATUS	REPORTED AT	MONITORING VIEW	HISTORY REPORTS
Changing the name for the time since d...	No results yet	No results yet	Foundry Test	No results yet
Custom scope monitor	No results yet	No results yet	Foundry Test	No results yet
Total Throughput V2 (greater than or e...	No results yet	No results yet	Foundry Test	No results yet
Total Throughput V2 (Less than or equa...	No results yet	No results yet	Foundry Test	No results yet

Related schedules

Expand all

Checks 3000

Foundry Test Monitoring View Passing (1) Failing (0)

Rules 2000

Compare

Use the **Compare** tab to compare two different datasets. Select the tab and choose a dataset to compare with. The **Compare** tab can be used in several ways:

- Compare two separate datasets to understand their differences.
- Compare a dataset to an older transaction of the same dataset to see how it changed over time.
- Compare the original version of a dataset with a different branch to see how merging that branch will affect the dataset.

i

Streaming datasets

In streaming datasets, the **Compare** tab will only appear when the view is set to **Archive**. You will then be able to compare the archive dataset with other non-

sets.

API Reference ↗

Send feedback

4. Preview table

Use the preview table to understand the structure of the data and to quickly explore the values in the dataset.

- i** By default, the preview table will show a limited sample of the data; the exact number of rows is displayed in the preview table header. However, any action taken on the data, such as filtering or sorting, will apply to the full dataset and increase the preview sample size. Depending on the number of rows, you may not see the entire dataset in the preview.

The preview table provides several useful capabilities:

- Select a column's menu to sort, filter, and generate charts over the column data.
- Select an individual cell to exclude or include only the selected value from the preview.
- Report and view issues on individual columns.
- Search for specific column names.

i Streaming datasets

The data preview table for streaming datasets provides a small sample of recently streamed rows. It will update automatically when set to **Live updates**. Sorting, filtering, and charting are only available when the page is set to **Archive** and will represent only the state of the archive dataset.

[API Reference ↗](#)

[Send feedback](#)

The screenshot displays the Foundry interface for a dataset named 'nyc_yellow_taxi_trips_manhattan'. The top section shows a preview of the first 13 rows of the dataset. The columns include 'end_borough', 'end_neighborhood', 'geohash', 'passenger_count', 'trip_time_in_secs', 'trip_distance', 'fare_amount', 'tip_amount', and 'total_amount'. A context menu is open over the 'end_neighborhood' column, showing options like 'Pin column', 'Encrypt column', 'Filter', 'Sort ascending', 'Sort descending', 'View stats', 'View cell content', 'Copy column name', and 'Expand'.

Below the preview, a summary table provides a breakdown of the data by neighborhood. The table has columns for 'end_neighborhood', 'count', 'length', and 'value'.

end_neighborhood	count	length	value
Midtown	16,719		
Upper East Side	13,273		
Upper West Side	7,937		
Chelsea	7,677		
Hell's Kitchen	4,844		
Theater District	3,309		
East Village	3,238		

Upload files manually

In Dataset Preview, you can upload files of the following types directly into a dataset: `.csv`, `.tsv`, `.xls`, `.xlsm`, and `.xlsx`.

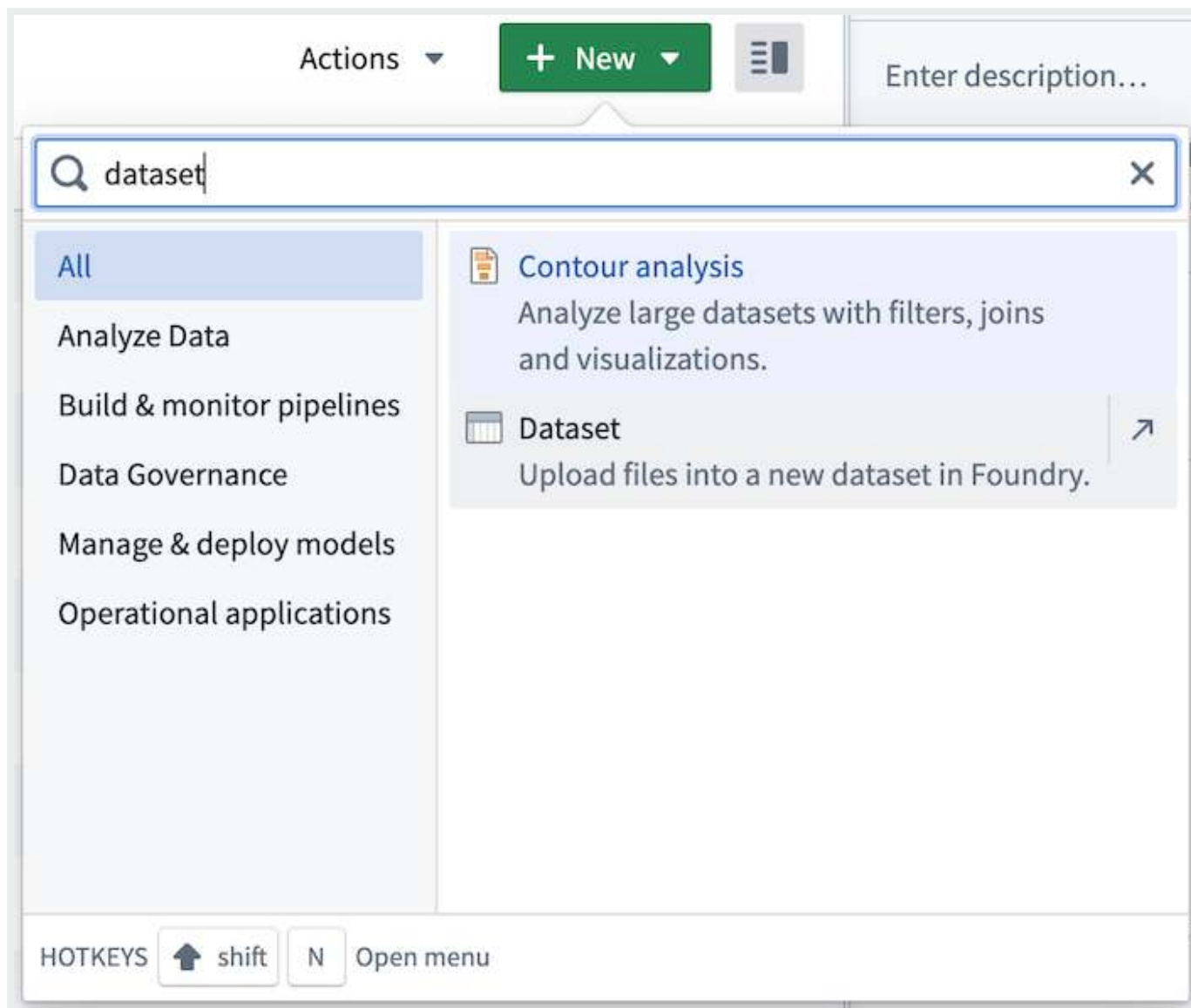
For `.csv` and `.tsv` files, Foundry will attempt to infer the schema of the new file. If the filename and schema of the new file are identical to a previous upload, you can [update data](#) in the existing dataset. If the filename is different from previous uploads, you can [append data](#) to an existing dataset.

The following steps apply to uploading all file types:

1. Navigate to your preferred folder and create a dataset.

[API Reference](#) ↗

[Send feedback](#)



2. Drag and drop the file into the dataset preview window.

5. Actions

The **All actions** dropdown menu provides quick access to Foundry tools and operations, allowing you to analyze, explore, transform, and manage the data. Some actions, such as **Analyze** (in Contour) and **Build**, are surfaced outside the actions menu for quick access.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Dataset Preview

The Dataset Preview application provides you with a variety of details of a given dataset, including metadata, build history, health, and more. Additional features are available for streaming datasets, including the ability to view information on streaming jobs and metrics to troubleshoot and debug stream performance.

The screenshot below displays the interface of the Dataset Preview application. The numbered sections are explained in more details in the following sections.



1. Dataset header

The header of the page identifies the selected dataset and provides basic information such as its name, display name (if existing), location, and the selected branch. The header also allows some file-related operations such as sharing, moving, renaming, and more.

[API Reference ↗](#)

2. Information panel

[Send feedback](#)

The information panel provides metadata about the dataset and some basic administrative operations. The panel is divided into three sections:

- **About:** Information including the time the dataset was created and updated, the users who created and last updated the dataset, the size of the table, any tools and input datasets used to create the data, tags, and more. An **Edit schema** view is also available under the **Updated by** section that will infer a schema for CSV and JSON files. Here, users can also apply additional parsing options to drop jagged rows, change encoding, or add additional columns like file path, byte offset for row, import timestamp, or row number. For other file types, schema edits can be made in the **Schema** section under the **Details** tab.
- **Columns:** Information on the different columns in the dataset, including the type of data, description, and data stats (percentage of null values, distributions and samples).
- **Schedules:** Information about any configured build [schedules](#) that will run to update the dataset.

3. Tab views

Preview

The **Preview** tab provides a sample of the data in a table and allows light interaction with the full dataset. Learn more about the preview table in [4. Preview table](#).

History

The **History** tab view provides historical job (build) information. A **Summary** view on the right side of the page shows aggregated information on job statuses over time.

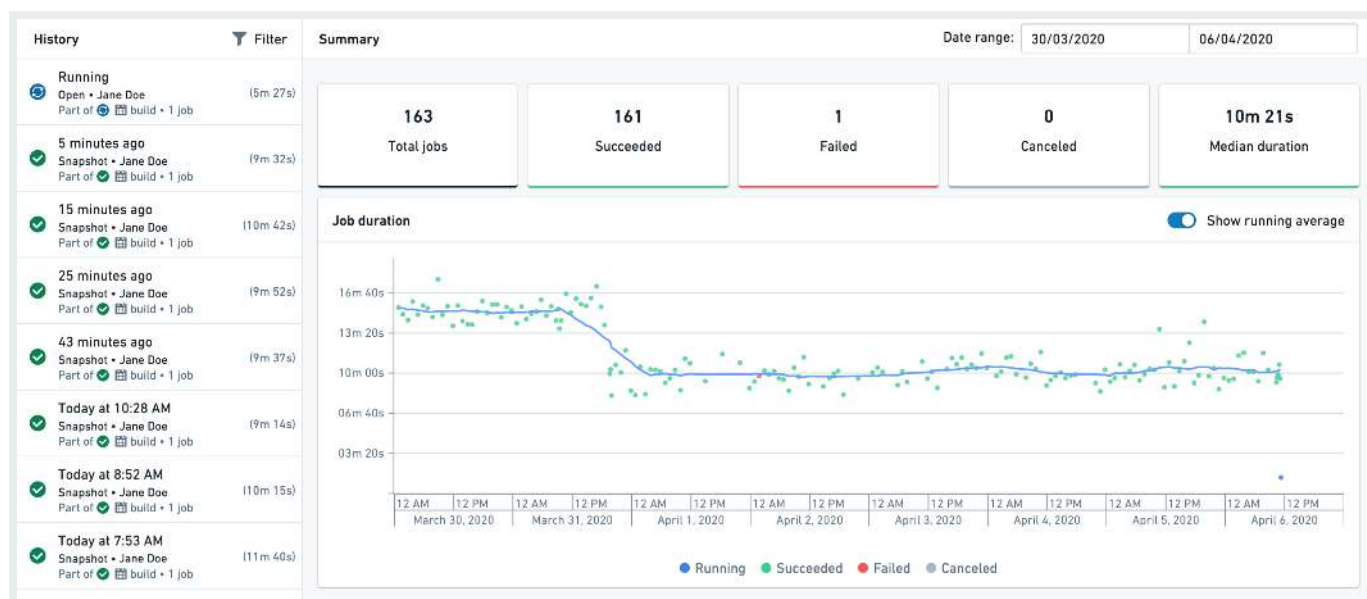
On the left panel, a list of jobs appears with their statuses and durations. Upon selection, a detailed **Job** view appears on the right showing detailed job information, including progress, specification, build logs, files and the resulting schema.

API Reference ↗

i Streaming datasets

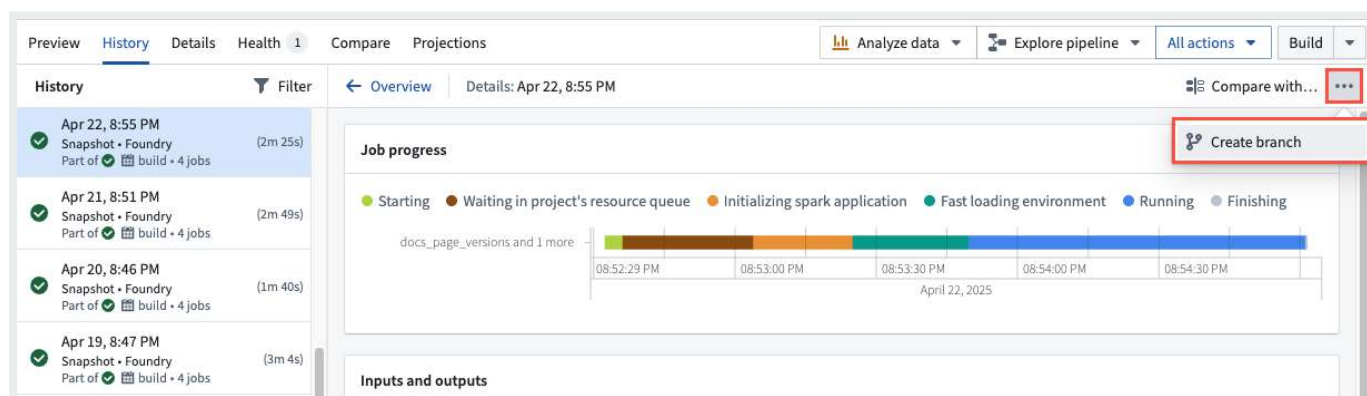
Send feedback

In streaming datasets, the **History** tab will only appear when the view is set to **Archive**. The **History** tab will show the archive transactions alongside the streaming jobs.



Create branches from historical transactions

You can use the **History** tab to create branches on historical transactions of your data that have not been deleted by a [retention policy](#). Choose a previous transaction from the left panel and select the ellipsis (...) icon to **Create branch**.



Details

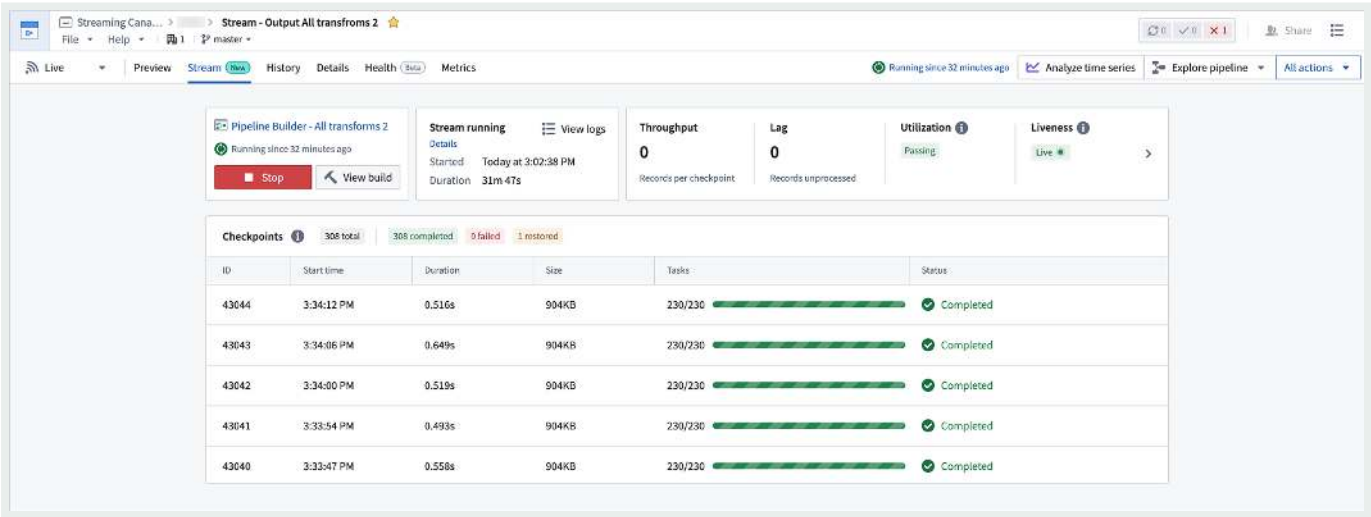
The **API Reference** provides additional technical information about the dataset, as well as some administrative operations:

[Send feedback](#)

- **Schema:** Provides full information on the table schema (column specifications) and allows you to edit the schema (if applicable).
- **Files:** Displays the list of files that make up the dataset and allows you to download them.
- **Job spec:** Displays the job specification containing essential information for the dataset to build.
- **Syncs:** Surfaces the status and details of data syncs to different databases. For some sync types, additional settings can be applied.
- **Custom metadata:** Allows you to add custom fields of information to the dataset. The fields added in this section are displayed in the information panel of the overview page.
- **Resource usage metrics:** Provides graphs and information on disk and Spark usage of the dataset over time.
- **Last run details (only for streams):** Shows detailed information about the latest stream run.

Stream (only for streaming datasets)

When the dataset is a streaming dataset, the **Stream** tab will show current and historical information on the streaming jobs. By changing the time period, you can explore the logs and details of jobs that streamed the dataset during that time.

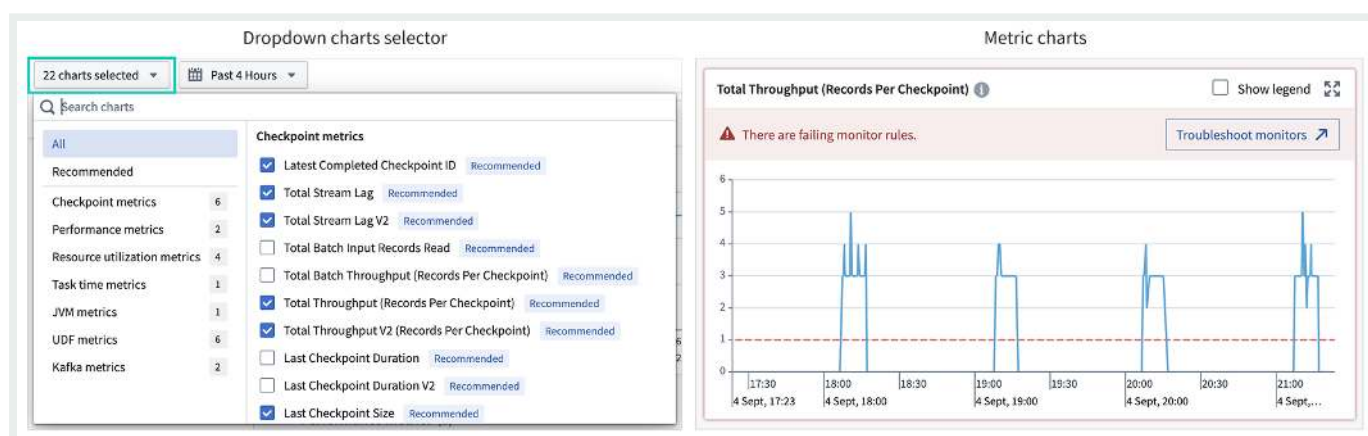
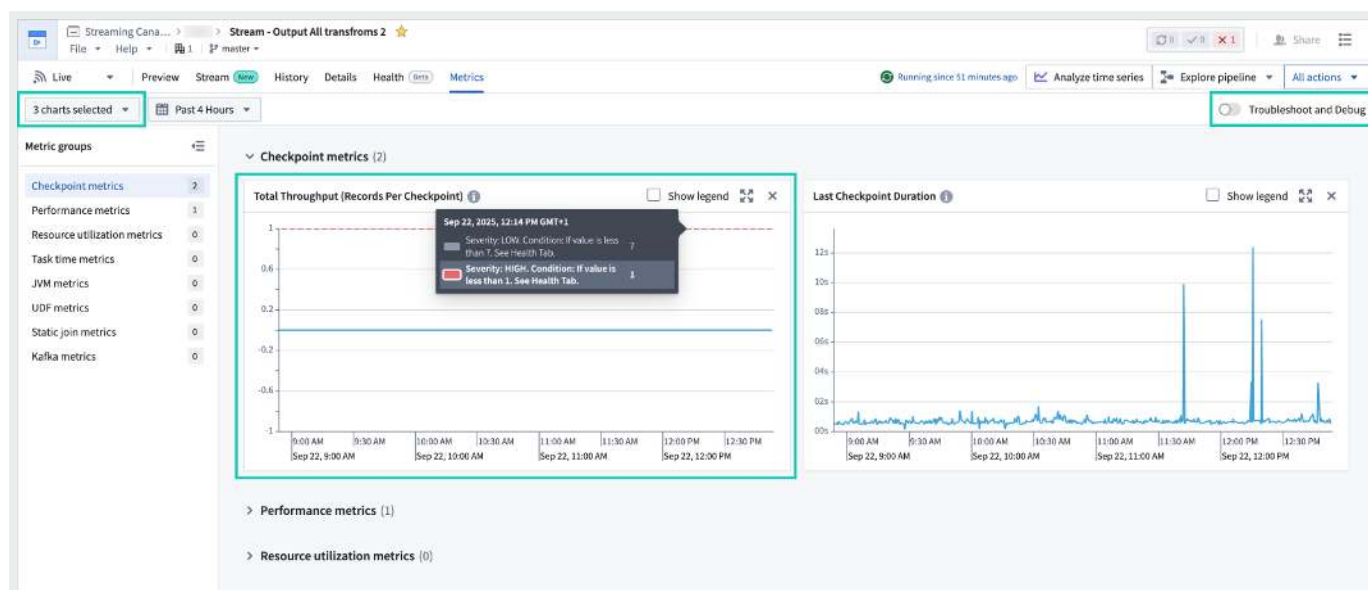


Metrics (only for streaming datasets)

With [API Reference ↗](#) a streaming dataset, the Metrics tab shows charts and related interactions for analyzing and troubleshooting streaming job performance. A dropdown menu for selecting metrics to visualize trends, with recommendations.

Send feedback

metrics highlighted within it. The metric charts can be expanded for detailed viewing and display thresholds and warning indicators with debug links.

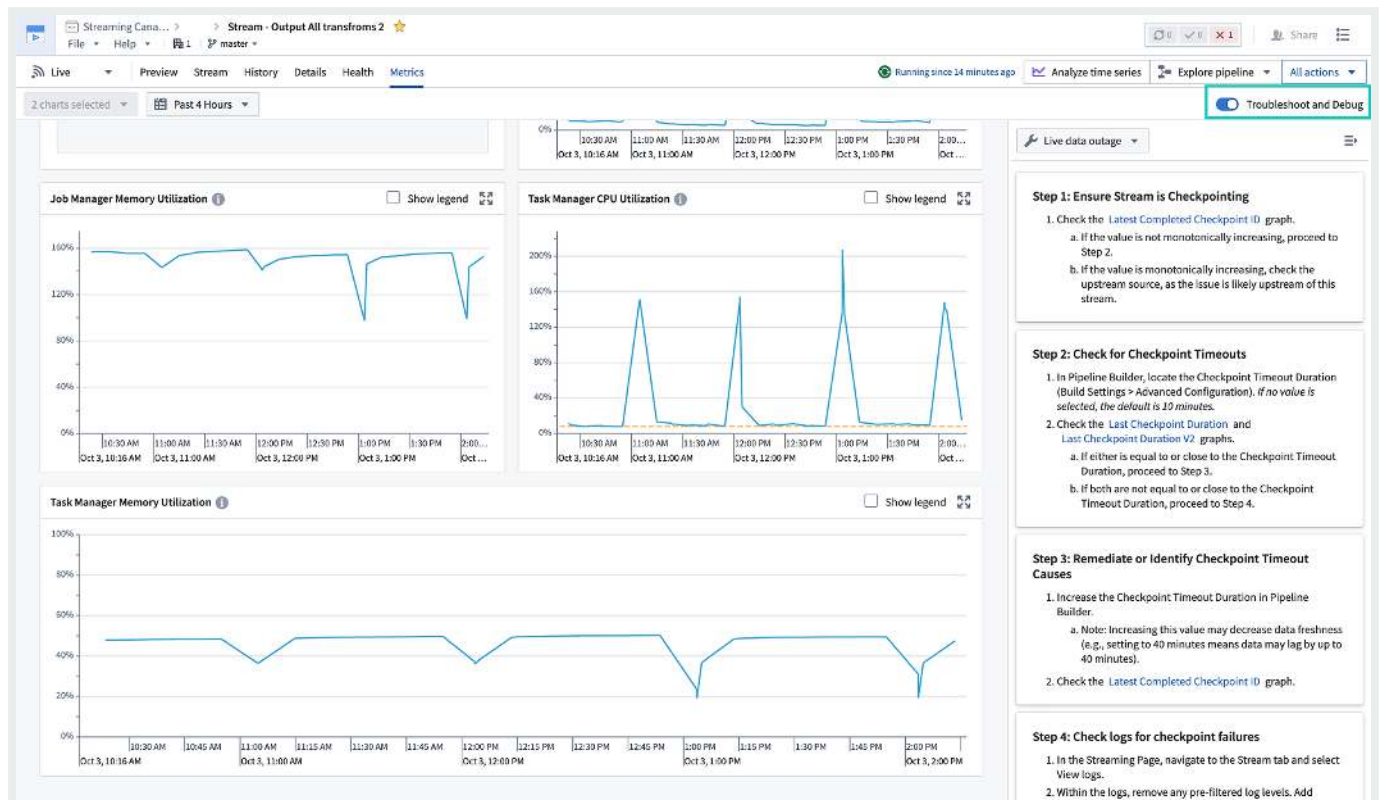


Troubleshoot and Debug mode

You can enable a dedicated troubleshooting mode using the toggle at the top right of the page. This mode provides a step-by-step walkthrough to debug stream outages. Select the in-line metrics tags from the right side panel to highlight the corresponding chart and easily locate the source of the issue.

API Reference ↗

Send feedback



Health

The **Health** tab provides tools to monitor [data health](#). The page displays health checks on the specific resource, monitoring rules on the resource grouped by specific monitoring view, and related schedule builds that affect the resource. Selecting any row reveals historical reports for the health checks and monitoring rules.

i Streaming datasets

In streaming datasets, the **Health** tab will only appear when the view is set to **Archive**. The checks will then refer to the archive dataset rather than the stream.

API Reference ↗

Send feedback

Pipelines > data > ... > nyc_yellow_taxi_trips_manhattan

FileHelp1master

Share

PreviewHistoryDetailsHealthCompare

Analyze dataExplore pipelineAll actionsBuild

Show failures only

Search monitors...Add checks

Checks

Checks 321

NAME	STATUS	REPORTED AT	MONITORING VIEW	HISTORY REPORTS
Allowed column values: dropoff	Failed	Aug 14, 2025, 13:10	No monitoring view	
Data Expectations Monitor	Passed	Aug 14, 2025, 13:09	No monitoring view	
Dataset file count	1	Aug 14, 2025, 13:09	No monitoring view	

Monitoring

Foundry Test Passing (28) Failing (2)

Rules 9000

+ Add rules

NAME	STATUS	REPORTED AT	MONITORING VIEW	HISTORY REPORTS
Changing the name for the time since d...	No results yet	No results yet	Foundry Test	No results yet
Custom scope monitor	No results yet	No results yet	Foundry Test	No results yet
Total Throughput V2 (greater than or e...	No results yet	No results yet	Foundry Test	No results yet
Total Throughput V2 (Less than or equa...	No results yet	No results yet	Foundry Test	No results yet

Related schedules

Expand all

Checks 3000

Foundry Test Monitoring View Passing (1) Failing (0)

Rules 2000

+ Add rules

Compare

Use the **Compare** tab to compare two different datasets. Select the tab and choose a dataset to compare with. The **Compare** tab can be used in several ways:

- Compare two separate datasets to understand their differences.
- Compare a dataset to an older transaction of the same dataset to see how it changed over time.
- Compare the original version of a dataset with a different branch to see how merging that branch will affect the dataset.

i

Streaming datasets

In streaming datasets, the **Compare** tab will only appear when the view is set to **Archive**. You will then be able to compare the archive dataset with other non-

API Reference ↗

Send feedback

4. Preview table

Use the preview table to understand the structure of the data and to quickly explore the values in the dataset.

- i** By default, the preview table will show a limited sample of the data; the exact number of rows is displayed in the preview table header. However, any action taken on the data, such as filtering or sorting, will apply to the full dataset and increase the preview sample size. Depending on the number of rows, you may not see the entire dataset in the preview.

The preview table provides several useful capabilities:

- Select a column's menu to sort, filter, and generate charts over the column data.
- Select an individual cell to exclude or include only the selected value from the preview.
- Report and view issues on individual columns.
- Search for specific column names.

i Streaming datasets

The data preview table for streaming datasets provides a small sample of recently streamed rows. It will update automatically when set to **Live updates**. Sorting, filtering, and charting are only available when the page is set to **Archive** and will represent only the state of the archive dataset.

[API Reference ↗](#)

[Send feedback](#)

The screenshot displays the Foundry interface for a dataset named 'nyc_yellow_taxi_trips_manhattan'. The top section shows a table with columns: end_borough, end_neighborhood, geohash, passenger_count, trip_time_in_secs, trip_distance, fare_amount, tip_amount, and total_amount. A context menu is open over the 'end_neighborhood' column, showing options like 'Pin column', 'Encrypt columns', 'Filter', 'Sort ascending', 'Sort descending', 'View stats', 'View cell content', 'Copy column name', and 'Expand'. Below the table, a histogram for the 'end_neighborhood' column is shown, with a bar chart and a table of counts for each neighborhood.

end_neighborhood	count
Midtown	16,719
Upper East Side	13,273
Upper West Side	7,937
Chelsea	7,677
Hell's Kitchen	4,844
Theater District	3,309
East Village	3,238

Upload files manually

In Dataset Preview, you can upload files of the following types directly into a dataset: `.csv`, `.tsv`, `.xls`, `.xlsm`, and `.xlsx`.

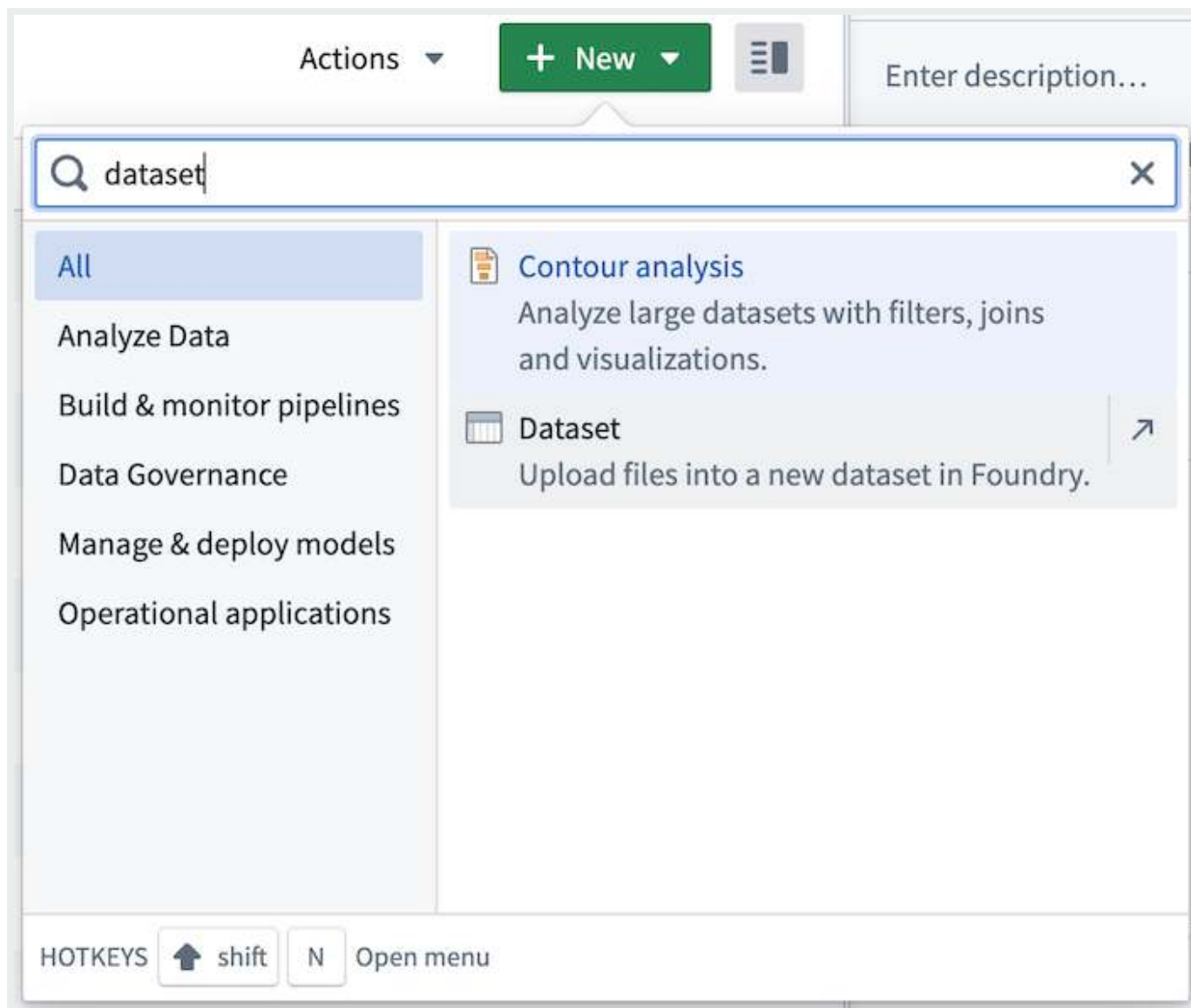
For `.csv` and `.tsv` files, Foundry will attempt to infer the schema of the new file. If the filename and schema of the new file are identical to a previous upload, you can [update data](#) in the existing dataset. If the filename is different from previous uploads, you can [append data](#) to an existing dataset.

The following steps apply to uploading all file types:

1. Navigate to your preferred folder and create a dataset.

[API Reference](#) ↗

[Send feedback](#)



2. Drag and drop the file into the dataset preview window.

5. Actions

The **All actions** dropdown menu provides quick access to Foundry tools and operations, allowing you to analyze, explore, transform, and manage the data. Some actions, such as **Analyze** (in Contour) and **Build**, are surfaced outside the actions menu for quick access.

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)

Data connectivity & integration / Dataset Preview / CSV parsing

CSV parsing

Foundry supports **CSV datasets**. These are datasets that contain files in the CSV format.

The CSV format can use different delimiters, quote characters, and escape characters. To manage this, you can define parameters that control how CSV files are parsed. These parameters are stored in the schema of a dataset. Foundry will use inference to suggest a sensible set of parameters for a given dataset, but results should be validated and changes made if necessary.

Parsing in Foundry

Foundry CSV datasets will generally have the `TextDataFrameReader` defined as their `dataFrameReaderClass` in the schema. This supports a set of custom parameters that can help deal with messy data effectively. At execution time, this delegates to the [Spark CSV DataFrameReader](#) for the best possible performance and reliability.

Configuration

In Foundry, you can view the schema on any dataset in the [Dataset Preview](#) application by navigating to the **Details** tab and selecting **Schema**. For more details on the schema, see the [Dataset](#) documentation.

CSV schemas can be manipulated in the **Edit Schema** UI, available from [Dataset Preview](#) when viewing the preview tab. This will help visualize the options available and how they affect the dataset. In cases where CSVs are particularly malformed, you may need to modify the schema to get the desired output.

API Reference ↗



Send feedback

TextDataFrameReader options

To manually configure the `TextDataFrameReader` options in the schema, you can navigate to the schema page in the **Details** tab of [Dataset Preview](#) and select **Edit**. At the bottom of the schema, there should be a section as follows:

```
1  "dataFrameReaderClass": "com.palantir.foundry.spark.input.TextDataFrameReader",
2  "customMetadata": {
3    "textParserParams": {
4      "parser": "CSV_PARSER",
5      "charsetName": "UTF-8",
6      "fieldDelimiter": ",",
7      "recordDelimiter": "\n",
8      "quoteCharacter": "\"",
9      "dateFormat": {},
10     "skipLines": 1,
11     "jaggedRowBehavior": "THROW_EXCEPTION",
12     "parseErrorBehavior": "THROW_EXCEPTION",
13     "addFilePath": false,
14     "addFilePathInsteadOfUri": false,
15     "addImportedAt": false,
16     "initialReadTimeout": "1 hour"
17   }
18 }
19 }
```

The following options are available for the `TextDataFrameReader` :

Property	Purpose	Accepted values	Required?
parser	The parser type to use.	CSV_PARSER, MULTILINE_CSV_PARSER, SIMPLE_PARSER, SINGLE_COLUMN_PARSER	Yes
nullValues	The values that should be parsed to null.	A list of strings	Yes
fieldDelimiter	The delimiter	A one-character string	

API Reference ↗

Send feedback or

Property	Purpose	Accepted values	Required?
	character for splitting a record into multiple fields.		
recordDelimiter	The end of line symbols for splitting a CSV file into multiple records.	A string ends with newline character	No, default to \n (newline) CSV_PARSER_MULTILINE_CSV_P
quoteCharacter	The quote character for CSV parsing.	A one-character string	No, default to " (doublequote)
dateFormat	Format strings for date parsing in certain columns.	A map that maps column names to JodaTime DateTimeFormat patterns	No, default to emp
skipLines	The number of lines to skip parsing at the start of each file.	A non-negative number	No, default to 0
jaggedRowBehavior	Behavior when there are more or fewer columns	THROW_EXCEPTION, DROP_ROW	No, defaults to THROW_EXCEPTIO
API Reference ↗ ☰			Send feedback

Property	Purpose	Accepted values	Required?
	than types specified in the header.		
parseErrorBehavior	Behavior when a value fails to parse into the requested type specified in the header.	THROW_EXCEPTION, REPLACE_WITH_NULL	No, defaults to THROW_EXCEPTIC
addFilePath	Each row is augmented by a file path.	Boolean	No, default to false
addImportedAt	Each row is augmented by the import time.	Boolean	No, default to false
initialReadTimeout	Limits the time the parser will wait to read the first row.	A human-readable duration	No, default to 1 hour

Spark CSV options

API Reference ↗

If you are familiar with the Spark CSV `DataFrameReader`, you can configure the `dataFrameReaderClass` to be `DataSourceDataFrameReader` and the `customMetadata`.

Send feedback

See the [Spark CSV DataFrameReader documentation ↗](#) for a list of supported options. You can add the configurations as key-value pairs like this:

```
1  "dataFrameReaderClass": "com.palantir.foundry.spark.input.DataSourceData
2  "customMetadata": {
3    "format": "csv",
4    "options": {
5      "unescapedQuoteHandling": "STOP_AT_DELIMITER",
6      "multiline": true,
7      ...
8    }
9  }
```

i Note that the schema options listed above are only applicable to datasets constructed from CSV files.

PREVIOUS
← Overview

NEXT
Time Travel →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— Cookies Settings

[API Reference ↗](#)

Send feedback

Data connectivity & integration / Dataset Preview / Time Travel

Time Travel [Beta]

Beta

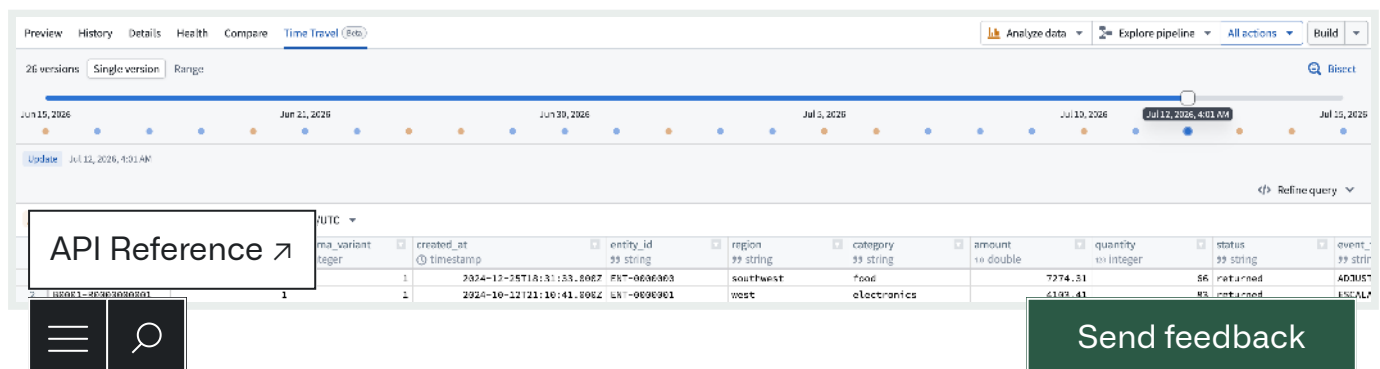
Time Travel is in the [beta](#) phase of development. Functionality may change during active development.

Time Travel in [Dataset Preview](#) lets you inspect how a [dataset](#) or [Iceberg table](#) changed across its committed versions. You can view the data and schema at an earlier point in time, view changes across a range of versions, and find the specific version where a row or value changed.

Time Travel builds its timeline from committed Catalog dataset transactions or Iceberg table snapshots. Each point on the timeline represents a committed version, in chronological order.

Open Time Travel

To use Time Travel, open a dataset or Iceberg table in Dataset Preview, then select the **Time Travel** tab. Time Travel provides two views: **Single version** and **Range**.



Preview History Details Health Compare Time Travel (800)

Analyze data Explore pipeline All actions Build

26 versions Single version Range

Jun 15, 2026 Jun 21, 2026 Jun 29, 2026 Jul 3, 2026 Jul 10, 2026 Jul 12, 2026, 4:01 AM Jul 15, 2026

Update Jul 12, 2026, 4:01 AM

Refine query

API Reference ↗

ma_variant	created_at	entity_id	region	category	amount	quantity	status	event
integer	timestamp	string	string	string	double	integer	string	string
1	2024-12-25T18:31:53.006Z	ENT-0696363	southwest	food	7374.31	56	returned	AD3015T
1	2024-10-12T21:10:41.006Z	ENT-0696361	west	electronics	6193.41	83	returned	153CALP

Send feedback

Explore a single version

Use the **Single version** view in Time Travel to see the data and schema for one committed version. As you move through the timeline, the preview updates to that version so you can see how the dataset looked at that point in time.

When relevant versions are loaded, you can also use the Left and Right arrow keys to move through them.

View changes across a time range

Use the **Range** view to see how data changed between an older baseline and a newer version in the selected time range.

Results can show:

- **Added** rows
- **Removed** rows
- **Modified** rows and cells, when a key column is available

A **Key column** allows Time Travel to match the same row across versions. With a stable, unique key, edits appear as modifications. Without one, changes to an existing row appear as a removal and an addition.

If the schema changed in the selected range, Time Travel also summarizes the columns that were added, removed, or changed.

26 versionsSingle versionRangeKey columnrow_id

Jun 15, 2026

Jun 21, 2026

Jun 28, 2026

Jul 5, 2026

Jul 12, 2026

Jul 19, 2026

Jul 26, 2026

Aug 2, 2026

Aug 9, 2026

Aug 16, 2026

Aug 23, 2026

Aug 30, 2026

Search for when data was introduced, changed, or removed

Snapshot

Jul 13, 2026, 4:04 AM

1

Execution time: 325.483ms

3,028,006 added16,927 removed19,528 modified

	diff_status	row_id	build_number	schema_variant	created_at	entity_id	region	category	amount	quantity	status
	string	string	string	string	string	string	string	string	string	string	string
1	Removed	80801-80809010397	1	1	2024-10-26 25:38:27	LOI-8010397	midwest	food	6026.72	15	shipped
2	Modified	80801-80806340383	1 + 24	1	2024-08-31 07:25:46	ENT-0340383	southwest	toys	6024.31 + 6471.83	18 + 11	shipped + pending
3	Modified	80801-8080937185	1 + 24	1	2024-08-29 02:25:54	ENT-0937185	north + northwest	furniture	1044.96 + 1363.82	24 + 79	pending + returned
4	Modified	80801-80802102859	1 + 24	1	2024-02-11 22:28:26	ENT-0102859	midwest	clothing	1636.2 + 4104.85	26 + 55	returned + cancelled
5	Removed	80801-80803686123	1	1	2024-04-26 23:58:22	ENT-8686123	north	furniture	2267.31	78	cancelled
6	Removed	80801-80803513335	1	1	2024-08-13 23:24:27	LOI-8513335	midwest	food	8257.34	19	cancelled
				1	2024-06-26 02:10:22	ENT-8077311	northwest	sports	2891.63	7	cancelled
				1	2024-12-02 21:52:21	ENT-8268263	southwest	electronics	4686.52	48	confirmed
				1	2024-10-03 05:12:03	ENT-0870284	northeast + west	sports	543.72 + 5345.96	38 + 23	delivered

API Reference ↗

Follow a SQL query through time

Send feedback

Select the **Refine query** option on the right to expose the SQL query behind a single-version preview. This is useful if you want to focus on a specific subset of rows, selected columns, a sort order, or an aggregate result.

After the query runs, Time Travel carries the query forward as you move through the timeline by updating its version reference automatically. This lets you follow the same analysis across earlier or later versions without rebuilding the SQL query.

Find when data changed

Select the **Bisect** option in the top right to search version history for the first time a change occurred. Bisect can be used to determine:

- When a row was added
- When a row was removed
- When a cell value changed

Bisect searches by a key column and, for cell-value changes, by a specific column and target value. When a matching version is found, you can navigate directly to it.

You can also right-click on a table cell to start a row or value bisect with the current context pre-filled.

 Bisect only searches across the versions currently loaded on the page.

[API Reference ↗](#)

[Send feedback](#)

Bisect

Jul 10, 2026Jul 12, 20Jul 13, 2026, 4:04 AMJul 15, 2026

3,000,000 added

19,922 removed

19,828 modified

category	amount
↕ string	↕ string
automotive	3156.77
food	1688.45
furniture	5788.62
toys	8004.68
electronics	7791.93
electronics	6156.47
tools	9140.12
toys	7540.48
books	7869.42
clothing	4725.15
sports	246.07
clothing	6693.27
books	9988.06

Bisect

X

Find

☒ When a row was added

☐ When a row was removed

☐ When a cell value changed

Key column

row_id

Key value

B0024-R0000009977

🕒 Row first appears in

ri.foundry.main.transaction.00000309-01e9-c3d6-8644-475ce6cfec8e

⌂

Jump to version

i You can only inspect versions where data is still available under the applicable [retention policy](#).

PREVIOUS
← CSV parsing

NEXT
FAQ →

© 2026 Palantir Technologies Inc. All rights reserved.

Cookies Statement

API Reference ↗

Send feedback

Data connectivity & integration / Dataset Preview / FAQ

Dataset Preview FAQ

The following are some frequently asked questions about Dataset Preview.

For general information, see the [Dataset Preview overview](#).

- [CSV fails to parse due to unmatched quote character](#)
- [Dataset fails to parse because some underlying CSVs have more columns](#)
- [Why does my downloaded dataset look strange when I open it in Microsoft Excel?](#)

CSV fails to parse due to unmatched quote character

When uploading a CSV with nested double-quotes and embedded newline (`\n`) characters within quoted fields, the schema inference will fail, and you cannot use the Schema Editor to create a valid schema.

To troubleshoot, perform the following steps:

1. Upload the CSV to a dataset and select **Edit Schema**.
2. Set the **Column** and **Quotes** properties.
3. Ignore the schema validation errors and select **Save without Validating**, available in the dropdown menu next to the **Save and Validate** option. This will create a schema with the correct column definitions.

4. [API Reference ↗](#), open the schema in edit mode.

5. Change `dataFrameReaderClass` to

```
palantir.foundry.spark.input.DataSourceDataFrameReader
```

6. Add the following to the `"customMetadata"` object:

Send feedback

```
"customMetadata": {  
  "format": "csv",  
  "options": {  
    "header": true,  
    "multiLine": true  
  }  
}
```

[Return to top](#)

Dataset fails to parse because some underlying CSVs have more columns

When a dataset is composed of multiple CSVs (for example, through a Data Connection **APPEND** transaction), and some of those CSVs contain more columns, the schema inference will fail. One option is to ignore jagged rows (such as rows that are missing certain columns). To do this, select **Edit schema**, expand the **Parsing options** section, and check **Ignore jagged rows**.

However, if you want to keep the jagged rows and specify a standardized schema for the dataset, then this section applies. If the conditions outlined in the **Assumptions** section below hold for your data, then the troubleshooting steps will produce a dataset with a standard schema *defined by the user*, in which jagged rows are autopopulated with **null** for the columns which they are missing.

Symptoms of parsing failures:

You may encounter an error message such as the following: "Could not load preview: Encountered an error parsing the input CSV data. Make sure all data is correctly formatted."

You may also encounter the following error message after selecting **Edit Schema** and then

S API Reference ↗ Your dataset failed to validate on x rows."

Assumptions:

[Send feedback](#)

1. You can define the desired schema, such as all column names and types that the dataset should possess.
2. Your schema enforces strict column ordering. For example, if you want the dataset to contain and show columns `{a, b, c}`, an underlying CSV can have a column structure like:

- `{a}`
- `{a, b}`

but cannot have a column structure like:

- `{b, a}`

3. An underlying CSV must have all columns preceding the n -th column if it has the $(n+1)$ th column. For example, if you want the dataset to contain and show columns `{a, b, c, d}`, an underlying CSV can have a column structure like:

- `{a}`
- `{a, b}`
- `{a, b, c}`

but cannot have a column structure like:

- `{b, c, d}`
- `{a, c, d}`
- `{a, b, d}`

Here is an example of a case in which the troubleshooting steps would be applicable and useful:

CSVs are regularly added to a dataset through `APPEND` transactions. One day, a new column is added and is the new last column in the CSV. In the dataset, rows from all previously-appended CSVs should have the new column, with field values autopopulated with `null` rather than being considered jagged.

The troubleshooting steps do not replicate the functionality of the `mergeSchema` option available for Parquet datasets (which are parsed with

[API Reference ↗](#) `ry.spark.input.ParquetDataFrameReader` as the `dataFrameReaderClass`). A user-written transform is required to repl on a raw CSV dataset.

Send feedback

To troubleshoot, perform the following steps:

1. In the **Details** tab, open the **Schema** tab, and select **Edit**
2. Modify the `fieldSchemaList` to ensure it includes all the columns that the dataset should possess. For example, if the dataset should have columns `{a, b, c}`, all with integer types, the `fieldSchemaList` may look like the following:

```
"fieldSchemaList": [  
  {  
    "type": "INTEGER",  
    "name": "a",  
    "nullable": null,  
    "userDefinedTypeClass": null,  
    "customMetadata": {},  
    "arraySubtype": null,  
    "precision": null,  
    "scale": null,  
    "mapKeyType": null,  
    "mapValueType": null,  
    "subSchemas": null  
  },  
  {  
    "type": "INTEGER",  
    "name": "b",  
    "nullable": null,  
    "userDefinedTypeClass": null,  
    "customMetadata": {},  
    "arraySubtype": null,  
    "precision": null,  
    "scale": null,  
    "mapKeyType": null,  
    "mapValueType": null,  
    "subSchemas": null  
  },  
  {  
    "type": "INTEGER",  
    "name": "c",  
    "nullable": null,  
    "userDefinedTypeClass": null,  
    "customMetadata": {},  
    "arraySubtype": null,  
    "precision": null,  
    "scale": null,  
    "mapKeyType": null,  
    "mapValueType": null,  
    "subSchemas": null  
  }  
]
```

[API Reference ↗](#)



[Send feedback](#)

```

    "userDefinedTypeClass": null,
    "customMetadata": {},
    "arraySubtype": null,
    "precision": null,
    "scale": null,
    "mapKeyType": null,
    "mapValueType": null,
    "subSchemas": null
  }
],

```

3. Change `"dataFrameReaderClass"` and its nested `customMetadata` object so that the end of your schema JSON looks like the following:

```

"dataFrameReaderClass": "com.palantir.foundry.spark.input.DataSourceDataFrameReader",
"customMetadata": {
  "format": "csv",
  "options": {
    "multiLine": true,
    "header": true,
    "mode": "PERMISSIVE"
  }
}
}

```

4. Select **Save**.

[Return to top](#)

Why does my downloaded dataset look strange when I open it in Microsoft Excel?

[API Reference ↗](#)

When exporting datasets from the platform, some files may appear strange when opened in Microsoft Excel. This issue has been observed in certain regions and is caused by the default delimiter used.

[Send feedback](#)

in Excel. To fix this issue, you will need to change the delimiter pattern in your export settings:

1. Open the file in Excel.
2. Select the **Data** tab in the ribbon.
3. Select the **Text to Columns** option in the **Data Tools** group.
4. In the **Convert text to columns wizard** window, select the **Delimited** option and then **Next**.
5. In the **Delimiters** section, select the delimiter you want to use (such as comma, semicolon, tab).
6. Select **Next** and then **Finish** to complete the process.

[Return to top](#)

PREVIOUS
← Time Travel

NEXT
Linters / Overview →

© 2026 Palantir Technologies Inc. All rights reserved.

[Cookies Statement ↗](#)

[Privacy Statement ↗](#)

[Terms of Use ↗](#)

— [Cookies Settings](#)

[API Reference ↗](#)

[Send feedback](#)